

RAPPORT TECHNIQUE FINAL

Plateforme Intelligente de Mobilité Urbaine

Optimisation de la mobilité urbaine à l'aide des données ouvertes

Information	À compléter
Formation	Mastère 2 Big Data & IA : SUP DE VINCI
Projet	Projet d'Études 2025/2026
Membres du groupe	Sanou Faye / Ben Mouh Kawtar / Marlene Jodom Matsoko / Salghat Issam
Date	14/06/2026

Table des matières

1. Introduction et contexte.....	4
1.1 Objectifs du projet.....	4
1.2 Périmètre du MVP.....	4
2. Organisation de l'équipe projet.....	4
3. Méthodologie projet.....	5
4. Planning détaillé.....	5
4.1. Planning.....	5
4.2. Jalons principaux.....	6
5. Gestion des risques.....	8
6. Architecture de la solution.....	8
6.1 . Vue d'ensemble des services.....	8
6.2. Flux de données.....	8
6.3. Technologies utilisées.....	9
7. Jeux de données utilisés.....	9
7.1. Justification du choix.....	9
7.2. Veille technologique.....	9
8. Pipeline ETL et qualité des données.....	10
8.1. Modes d'exécution.....	10
8.2. Couche qualité des données.....	10
8.3. Modèle de données (PostgreSQL).....	10
8.4. Idempotence et rétention.....	11
9. Modélisation prédictive.....	11
9.1. Architecture hybride.....	11
9.2. Évaluation (backtest temporel).....	11
10. API REST : endpoints et sécurité.....	12
10.1 Endpoints disponibles.....	12
10.2 Démonstration de l'API.....	12
10.3. Sécurité.....	13
11. Dashboard interactif.....	13
11.1. Vue d'ensemble.....	14
11.2. Itinéraire intelligent multi-trajets (choix utilisateur).....	14
11.3. Carte thermique de congestion.....	15

11.4. Environnement.....	17
11.5. Corrélation trafic ↔ pollution.....	18
11.6. Comparaison de zones.....	19
11.7. Simulation what-if (pilotage gestionnaire).....	19
11.8. Historique ETL.....	21
11.9. Synthèse des onglets.....	21
12. Crowdsourcing citoyen.....	22
13 Résultats et métriques de performance.....	22
13.1. Précision de prédiction (exemple).....	22
11.2. KPIs du projet.....	22
12. Green IT et observabilité.....	23
12.1. Recommandations écologiques.....	23
12.2. Green IT par conception.....	23
12.3. Observabilité.....	23
13. Accessibilité et Réglementation.....	23
14. Qualité logicielle et CI/CD.....	24
14.1. Tests automatisés.....	24
14.2. Linting et formatage.....	24
14.3. CI/CD GitHub Actions.....	24
15. Documentation utilisateur.....	24
15.1. Prérequis.....	24
15.2. Installation et démarrage.....	24
15.3. Utilisation.....	24
15.4. Commandes utiles.....	25
16. Gestion des coûts et Limites et perspectives d'évolution.....	25
16.1. Estimation cloud (production, faible charge).....	25
16.2. Limites actuelles.....	25
16.3. Perspectives.....	25
16.4. Plan de Continuité et de Reprise d'Activité.....	26
17. Analyse critique.....	26
18. Conclusion.....	27

1. Introduction et contexte

Les grandes métropoles sont confrontées à une crise chronique de la mobilité : embouteillages quotidiens, retards des transports publics, pollution atmosphérique et insatisfaction croissante des usagers. L'émergence des données ouvertes (open data) et des technologies Big Data & IA ouvre de nouvelles opportunités pour analyser, prévoir et optimiser les flux urbains en temps réel.

Problématique : comment exploiter des données ouvertes hétérogènes pour construire une plateforme intelligente capable d'anticiper la congestion, d'optimiser les itinéraires, de formuler des recommandations écologiques et de donner la parole aux citoyens ?

1.1 Objectifs du projet

- Réduire la congestion urbaine en anticipant les flux de trafic (modèles prédictifs hybrides).
- Optimiser les itinéraires en tenant compte des perturbations prédites.
- Intégrer des données environnementales (qualité de l'air, météo) pour des recommandations écologiques.
- Fournir aux citoyens une interface interactive et un système de signalement participatif.
- Donner aux gestionnaires urbains des outils de simulation et de comparaison.
- Respecter les bonnes pratiques d'ingénierie (tests, CI/CD, containerisation Docker).

1.2 Périmètre du MVP

Le MVP couvre la ville de Paris (5 zones simulées), avec extension possible à tout territoire disposant d'un portail OpenDataSoft. Les données peuvent être générées en mode simulation (offline) ou récupérées depuis des API open data réelles.

2. Organisation de l'équipe projet

Le projet a été réalisé par une équipe de quatre étudiants aux profils complémentaires. Cette répartition des responsabilités a permis de couvrir l'ensemble du cycle de vie du projet, depuis la collecte des données jusqu'à la mise à disposition d'outils d'aide à la décision.

Membre	Rôle principal	Missions réalisées	Charge estimée
Faye Sanou	Data Engineer	Collecte des données open data, développement du pipeline ETL, intégration PostgreSQL	25%
Salghat Issam	Data Scientist	Préparation des données, développement des modèles prédictifs ARIMA et MLP, évaluation des performances	25%

Marlene Jodom Matsoko	Développeuse Full Stack	Développement de l'API REST, intégration des services backend	25%
Ben Mouh Kawtar	Data Analyst & BI	Conception du tableau de bord Streamlit, visualisations, analyse des indicateurs métier Des réunions hebdomadaires ont été organisées afin d'assurer le suivi des tâches, la coordination des développements et la résolution des problèmes techniques rencontrés.	25%

3. Méthodologie projet

Méthodologie adoptée

Le projet a été conduit selon une approche Agile inspirée de Scrum. Cette méthode a permis de développer progressivement les différentes composantes de la plateforme tout en assurant une validation régulière des fonctionnalités.

Le travail a été structuré en plusieurs sprints :

Sprint 1 : analyse du besoin et identification des sources de données.

Sprint 2 : conception et développement du pipeline ETL.

Sprint 3 : mise en place de l'entrepôt de données PostgreSQL.

Sprint 4 : développement des modèles de prédiction du trafic.

Sprint 5 : développement de l'API REST.

Sprint 6 : conception du tableau de bord décisionnel.

Sprint 7 : phase de tests, optimisation et documentation.

Cette organisation a permis d'obtenir rapidement un MVP fonctionnel puis d'améliorer progressivement la solution.

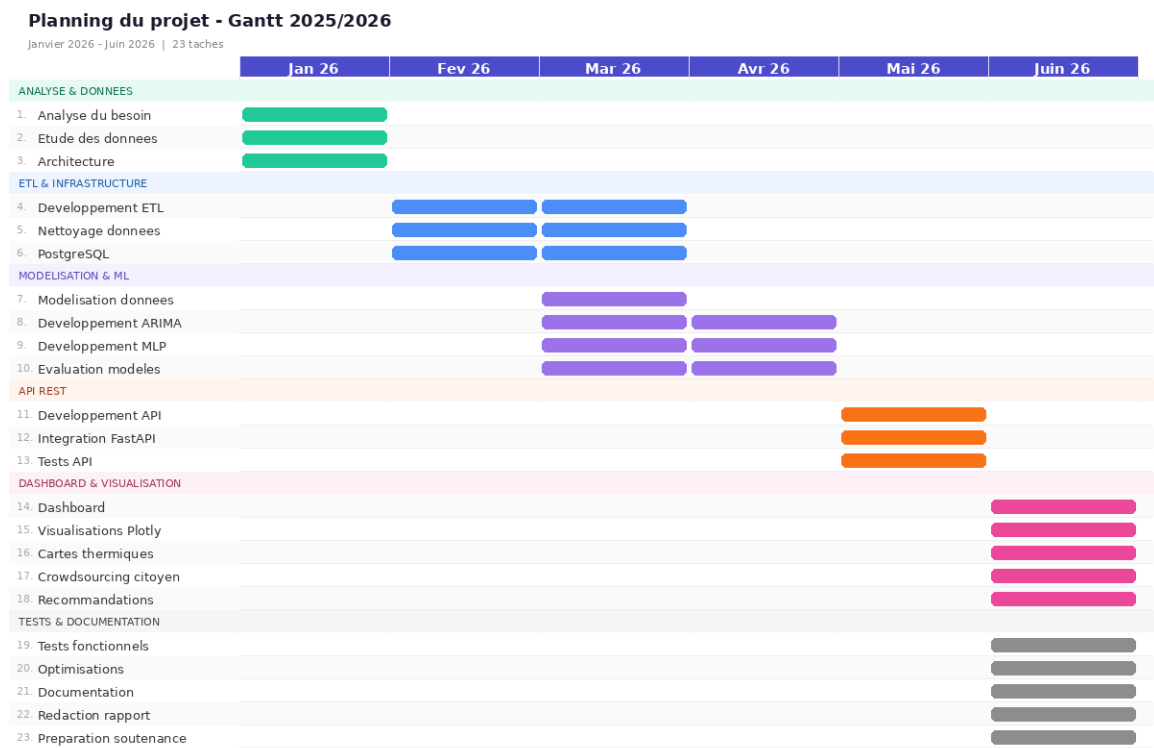
4. Planning détaillé

4.1. Planning

Le projet s'est déroulé de janvier à juin 2026.

Période	Activité
Janvier	Analyse du besoin, étude des données open data
Février	Développement du pipeline ETL
Mars	Mise en place de PostgreSQL et modélisation des données
Avril	Développement des modèles prédictifs
Mai	Développement API et Dashboard
Juin	Tests, optimisation, documentation et préparation de la soutenance

Diagramme Gantt



4.2. Jalons principaux

J1 - Validation des besoins et de l'architecture

Date : Fin janvier 2026

J2 - Pipeline ETL opérationnel

Date : Fin février 2026

J3 - Base PostgreSQL et données intégrées

Date : Mi-mars 2026

J4 - Modèles prédictifs validés

Date : Fin avril 2026

J5 - API REST fonctionnelle

Date : Mi-mai 2026

J6 - Dashboard interactif terminé

Date : Fin mai 2026

J7 - MVP finalisé

Date : Début juin 2026

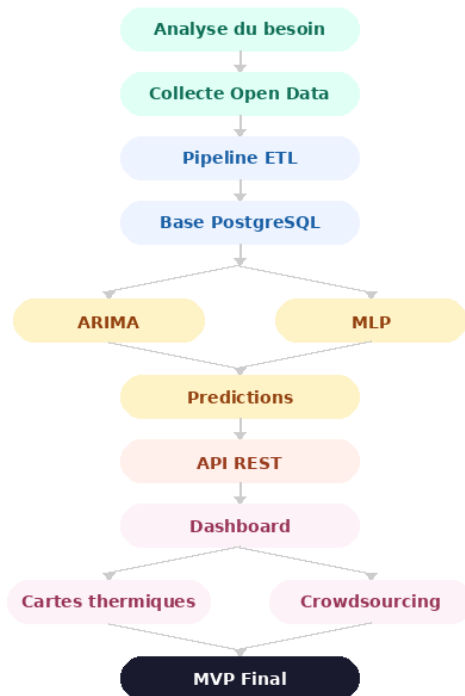
J8 - Rapport et soutenance finalisés

Date : Fin juin 2026

Diagramme des dépendances

Diagramme des dépendances

De l'analyse du besoin au MVP Final



5. Gestion des risques

Tout projet Big Data présente des risques techniques et organisationnels. Une analyse préventive a été réalisée afin d'anticiper les difficultés potentielles.

Risque	Impact	Probabilité	Solution retenue
Indisponibilité des API Open Data	Élevé	Moyen	Utilisation de jeux de données sauvegardés
Faible qualité des données	Moyen	Élevé	Mise en place d'étapes de nettoyage
Performance insuffisante des modèles	Élevé	Moyen	Comparaison de plusieurs modèles
Retards de développement	Moyen	Moyen	Répartition équilibrée des tâches
Perte de données	Élevé	Faible	Sauvegardes régulières

6. Architecture de la solution

La plateforme repose sur une architecture microservices containerisée (Docker Compose), composée de quatre services indépendants communiquant via le réseau Docker interne.

6.1. Vue d'ensemble des services

Couche	Service	Rôle
Données	PostgreSQL + PostGIS	Base spatiale : trafic, environnement, signalements, historique ETL
Collecte	Pipeline ETL (Python)	Collecte (ODS, Open-Meteo, simulation), nettoyage, chargement
Logique	API Flask	REST : données, prédictions, itinéraires, simulation, métriques
Visualisation	Dashboard Streamlit	Interface 7 onglets interactifs

6.2. Flux de données

Le flux suit un schéma linéaire **ETL** → **Base** → **API** → **Dashboard** : extraction des sources, transformation (qualité), chargement (UPSERT), exposition via l'API, puis visualisation interactive.

6.3. Technologies utilisées

Domaine	Outil	Justification
Base de données	PostgreSQL + PostGIS 16	Requêtes spatiales natives, robustesse, open source
Backend API	Flask + Gunicorn	Léger, rapide, adapté aux APIs REST
Pipeline ETL	Python + SQLAlchemy	Richesse de l'écosystème data
Modélisation	statsmodels + scikit-learn	ARIMA éprouvé + MLP pour les résidus
Visualisation	Streamlit + Plotly + pydeck	Dashboard interactif sans JavaScript
Itinéraires	Nominatim + OSRM	Géocodage et calcul d'itinéraires open source
Containerisation	Docker + Docker Compose	Portabilité, déploiement reproductible
Monitoring	Prometheus Client	Métriques HTTP exposées sur /metrics
Validation	Pydantic v2	Validation stricte des entrées de l'API
CI/CD	GitHub Actions	Lint, tests, audit sécurité à chaque push

7. Jeux de données utilisés

Conformément au cahier des charges, le projet exploite **au moins 3 catégories de données ouvertes**.

Catégorie	Source	Type	Licence
Trafic routier	OpenDataSoft (opendata.paris.fr)	API REST Explore v2.1	Open Data Commons
Trafic simulé	Générateur interne (mode sample)	Algorithme paramétré	Données de test
Météo	Open-Meteo	API REST (sans clé)	CC BY 4.0
Qualité de l'air	Open-Meteo Air Quality	API REST (sans clé)	CC BY 4.0
Crowdsourcing	Signalements citoyens	POST API interne	:

7.1. Justification du choix

Open-Meteo est retenu pour l'absence de clé API (déploiement offline aisé), sa couverture mondiale et la richesse de ses paramètres. OpenDataSoft est choisi pour sa compatibilité avec la plupart des portails open data français (API unifiée Explore v2.1).

7.2. Veille technologique

Une veille technologique a été réalisée afin d'identifier les solutions les plus adaptées aux besoins du projet.

Comparaison des bases de données

Solution	Avantages	Limites
PostgreSQL	Robuste, open source,	Moins flexible que NoSQL

	compatible PostGIS	
MongoDB	Flexible pour données semi-structurées	Cohérence transactionnelle plus complexe

PostgreSQL a été retenu en raison de sa stabilité et de sa capacité à gérer efficacement les données géographiques.

Comparaison des modèles prédictifs

Modèle	Avantages	Limites
ARIMA	Interprétable, efficace sur séries temporelles	Difficulté avec les relations non linéaires
LSTM	Très performant sur données complexes	Nécessite davantage de données
MLP	Simplicité de mise en œuvre	Sensible à la qualité des données

L'approche hybride ARIMA + MLP a été retenue afin de combiner les avantages des deux méthodes.

8. Pipeline ETL et qualité des données

Le pipeline ETL collecte, nettoie et charge les données dans PostgreSQL. Chaque exécution est historisée dans la table **etl_runs**.

8.1. Modes d'exécution

Mode	Description
sample	Génère des données réalistes simulées (trafic + environnement). 100% offline.
open-meteo	Récupère météo et qualité de l'air réelles depuis Open-Meteo (sans clé).
ods	Récupère des relevés trafic réels depuis un portail OpenDataSoft (paramétrable).

8.2. Couche qualité des données

- **Horodatages manquants** : les lignes sans timestamp sont rejetées.
- **Contrôle de plages** : valeurs aberrantes clippées (ex. congestion $\in [0,1]$, vitesse $\in [0,200]$).
- **Valeurs obligatoires** : pour le trafic, l'indice de congestion est requis.
- **Déduplication** : doublons (area_id + ts + source) éliminés, dernière valeur conservée.

Un rapport qualité (lignes entrantes/sortantes, doublons, anomalies) est journalisé pour chaque lot.

8.3. Modèle de données (PostgreSQL)

Table	Rôle
-------	------

areas	Référentiel des zones géographiques (avec géométrie PostGIS)
traffic_observations	Historique des mesures de trafic (vitesse, volume, congestion)
environment_observations	Historique environnemental (PM2.5, NO ₂ , O ₃ , AQI, météo)
crowd_reports	Signalements citoyens (catégorie, sévérité, statut)
etl_runs	Historique des exécutions ETL (observabilité)

8.4. Idempotence et rétention

Toutes les insertions utilisent des requêtes UPSERT (INSERT ... ON CONFLICT DO UPDATE) : relancer l'ETL ne crée pas de doublons. Une politique de purge à 90 jours est documentée dans le schéma SQL.

9. Modélisation prédictive

Le modèle de prédiction du trafic est **hybride ARIMA + MLP** : ARIMA capte la composante temporelle linéaire, le MLP apprend les résidus non-linéaires à partir de features calendaires.

9.1. Architecture hybride

Étape 1 : ARIMA(2,0,2) : ajusté sur la série de congestion des 7 derniers jours, il capte la tendance et l'autocorrélation.

Étape 2 : MLP sur les résidus : un réseau (couches 32 et 16) modélise les patterns non-linéaires à partir de 12 features calendaires.

Feature	Description
hour, dow, minute	Heure, jour de la semaine, minute
sin/cos (heure, jour, minute)	Encodage cyclique du temps
is_weekend	1 si samedi ou dimanche
is_holiday	1 si jour férié français (computus algorithmique)
is_rush	1 si heure de pointe (7-9h, 17-19h)

Avantage clé : ces features sont connues à l'avance (calendrier), donc valides aussi bien à l'entraînement qu'à l'inférence sur des pas futurs.

Étape 3 : Prédiction finale : $\text{congestion}(t) = \text{ARIMA}(t) + \text{MLP_résidu}(t)$, bornée à $[0,1]$.

9.2. Évaluation (backtest temporel)

Le modèle est évalué par un split temporel 80/20 et comparé à une baseline naïve saisonnière, via l'endpoint GET /api/v1/predictions/eval.

Métrique	Description
MAE	Erreur absolue moyenne
RMSE	Racine de l'erreur quadratique moyenne
MAPE	Erreur en pourcentage (robuste aux valeurs proches de 0)

10. API REST : endpoints et sécurité

L'API est construite avec Flask (Gunicorn). Elle implémente des pratiques de production : validation stricte, erreurs JSON uniformes, logs structurés, rate limiting, CORS paramétrable et métriques Prometheus.

10.1 Endpoints disponibles

Méthode	Endpoint	Description
GET	/health	Santé du service
GET	/metrics	Métriques Prometheus (latence, compteurs)
GET	/api/v1/areas	Liste des zones géographiques
GET	/api/v1/traffic/latest	Dernière mesure de trafic (param : area_id)
GET	/api/v1/traffic/series	Série temporelle trafic (params : area_id, hours)
GET	/api/v1/traffic/overview	Congestion récente par zone (carte thermique)
GET	/api/v1/environment/latest	Dernières mesures environnementales
GET	/api/v1/environment/series	Série temporelle environnement
GET	/api/v1/predictions	Prévisions de congestion (params : horizon, pas)
GET	/api/v1/predictions/eval	Backtest MAE/RMSE/MAPE vs baseline
GET	/api/v1/recommendations	Recommandations mobilité et écologiques
GET	/api/v1/simulation	Simulation what-if (param : reduction_pct)
POST	/api/v1/route	Itinéraires A→B classés par durée prévue
GET	/api/v1/crowd_reports	Signalements citoyens paginés
POST	/api/v1/crowd_reports	Créer un signalement (validé Pydantic)
GET	/api/v1/etl/runs	Historique des exécutions ETL

10.2 Démonstration de l'API

Pretty-print ☐

```
{ "status": "ok", "ts": "2026-06-14T13:20:16.060159+00:00" }
```

Fig. 1 : GET /health : réponse JSON confirmant que l'API est opérationnelle.

Pretty-print ☐

```
{"area_id":"paris_centre","models":{"arima+mlp":{"mae":0.145673327742253,"mape_pct":54.414310734745385,"n":409,"rmse":0.17670488711569401},"seasonal_naive":{"mae":0.88652516293783088,"mape_pct":50.127165204137332,"n":409,"rmse":0.10456339297305183}},"step_minutes":5,"test_size":409,"train_size":1637}
```

Fig. 2 : GET /api/v1/predictions/eval : métriques MAE/RMSE/MAPE du modèle hybride comparé à la baseline naïve saisonnière.

10.3. Sécurité

Mesure	Détail
Validation	Pydantic v2 valide chaque champ (type, plage, catégorie). Erreurs 400 détaillées.
Rate limiting	200 req/min par défaut ; écritures limitées (20 req/min).
API Key	Optionnelle : si API_KEY défini, en-tête X-API-Key exigé sur les écritures.
CORS	Origines autorisées paramétrables via CORS_ORIGINS.
Pagination	limit + offset sur les endpoints de liste.

11. Dashboard interactif

Le dashboard (Streamlit) est organisé en 7 onglets, accessibles via <http://localhost:8501>. Il consomme l'API REST et ne contient aucune logique métier propre.

11.1. Vue d'ensemble

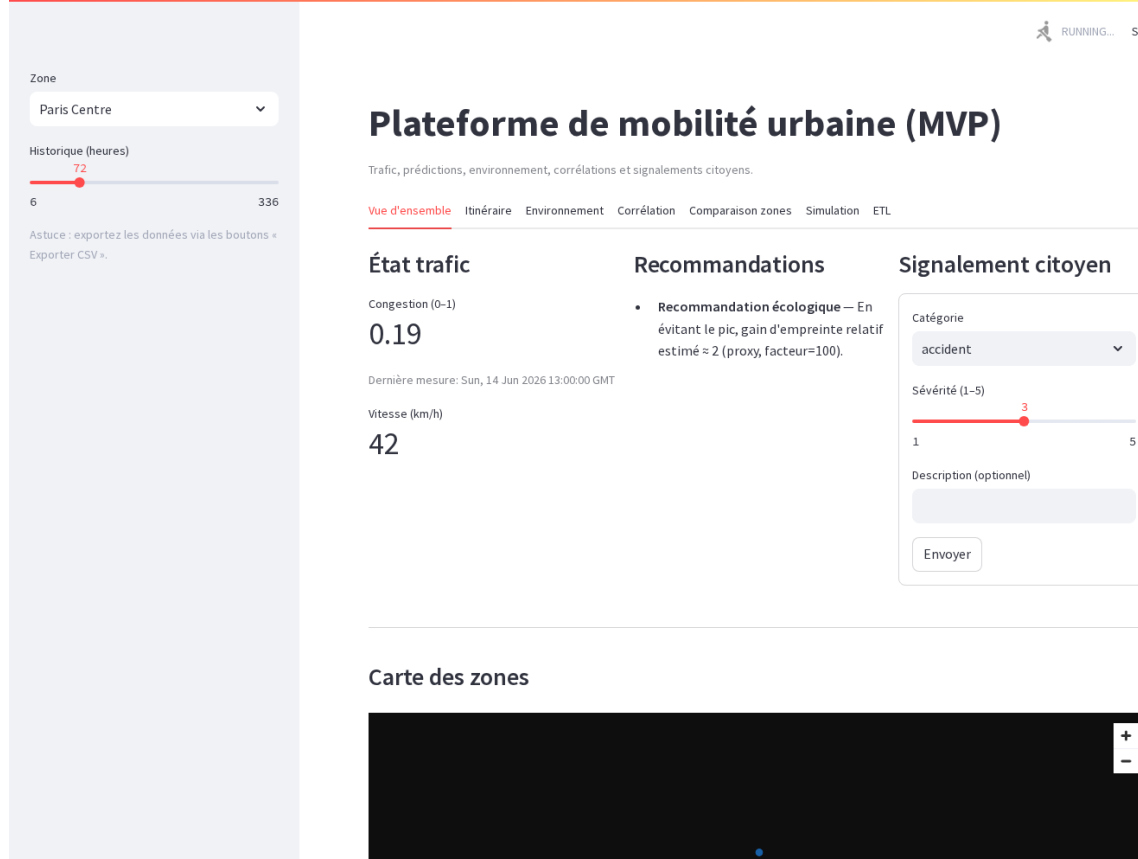


Fig. 3 : Onglet Vue d'ensemble : état du trafic (KPIs), recommandations, carte des zones et prédiction de congestion.

11.2. Itinéraire intelligent multi-trajets (choix utilisateur)

Fonctionnalité répondant à l'exigence « **optimiser les itinéraires** ». L'API calcule plusieurs itinéraires alternatifs (OSRM), évalue pour chacun la congestion **prédite** à l'heure de passage, puis les classe par durée prévue (perturbations incluses). **L'utilisateur choisit librement** son itinéraire ; le plus rapide est marqué d'un éclair.

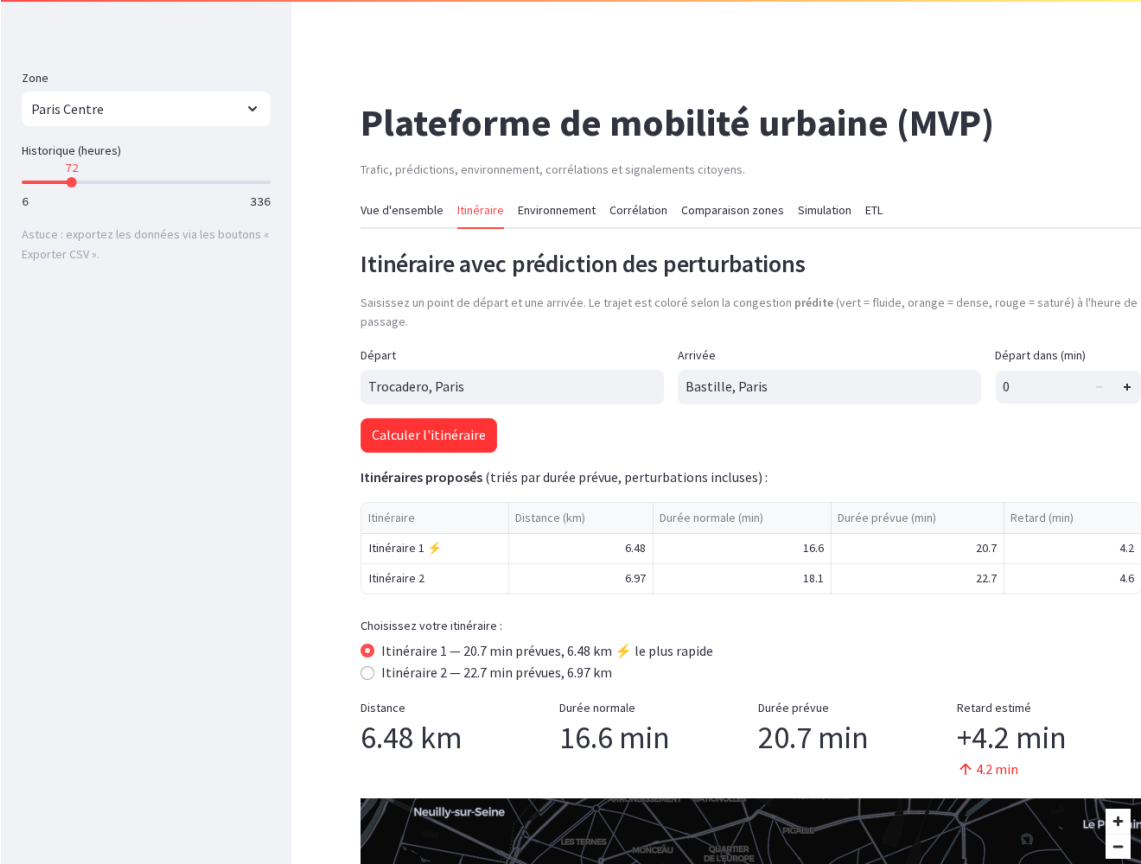


Fig. 4 : Onglet Itinéraire : itinéraires alternatifs comparés (durée normale vs durée prévue), sélecteur de choix et trajet coloré selon la congestion prédite.

11.3. Carte thermique de congestion

Répond à l'exigence « **cartes thermiques** ». La congestion récente de chaque zone est agrégée en carte de chaleur, exploitant la couche spatiale PostGIS.

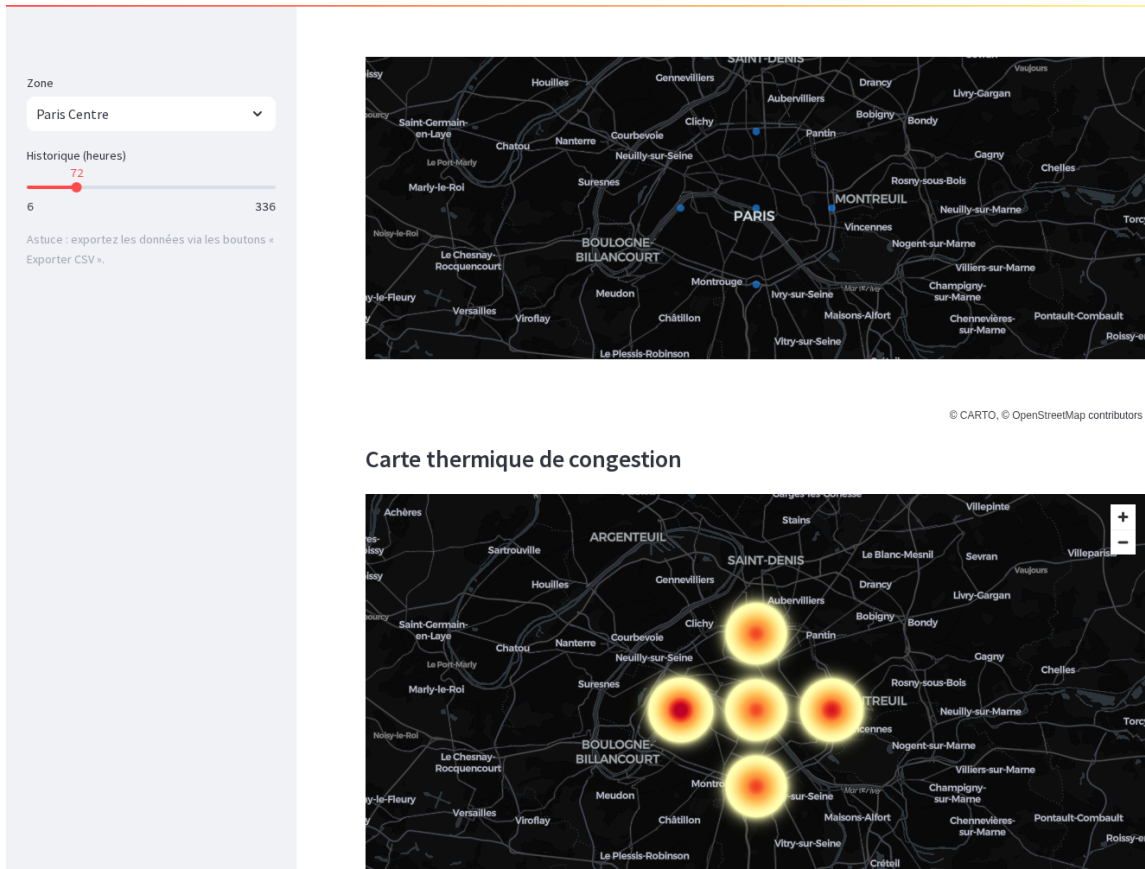
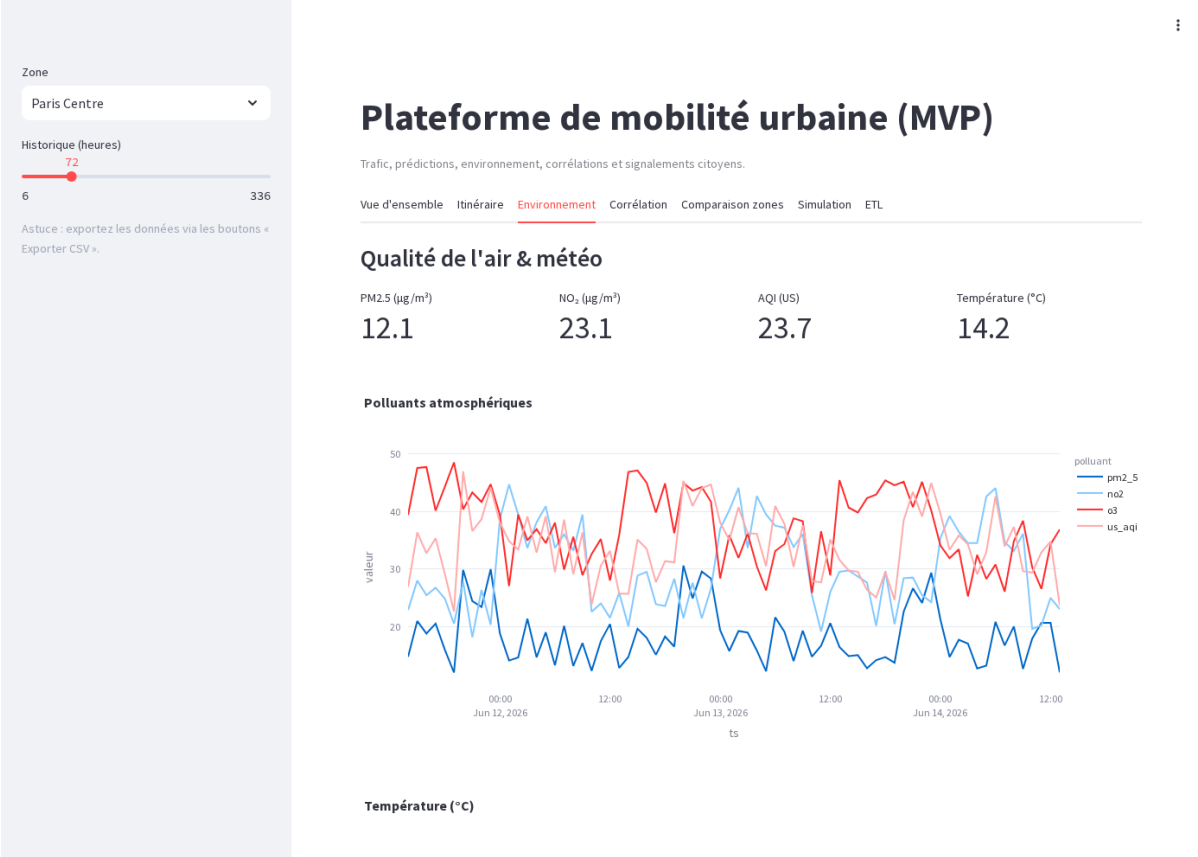


Fig. 5 : Carte thermique : zones de Paris en dégradé selon la congestion (points chauds = zones saturées).

11.4. Environnement



11.5. Corrélation trafic ↔ pollution

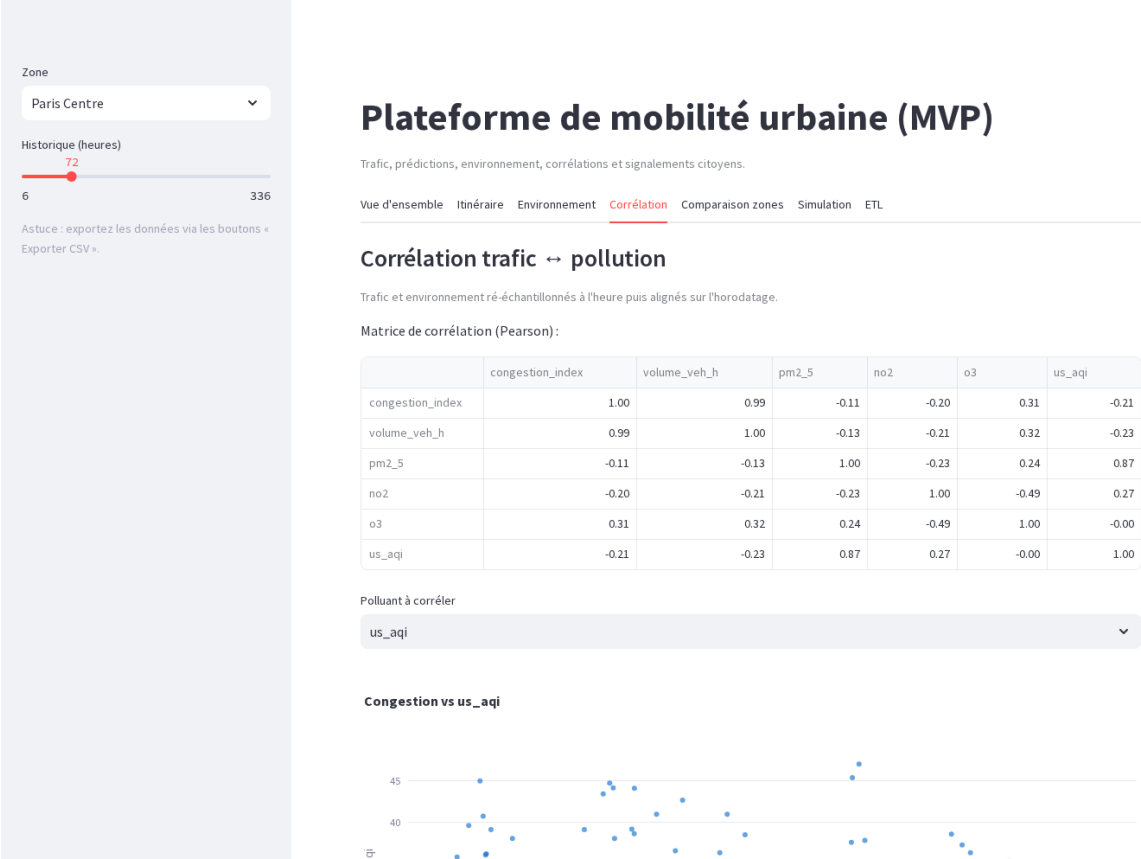


Fig. 7 : Onglet Corrélation : matrice de Pearson et nuage de points congestion vs polluant (données ré-échantillonnées à l'heure).

11.6. Comparaison de zones

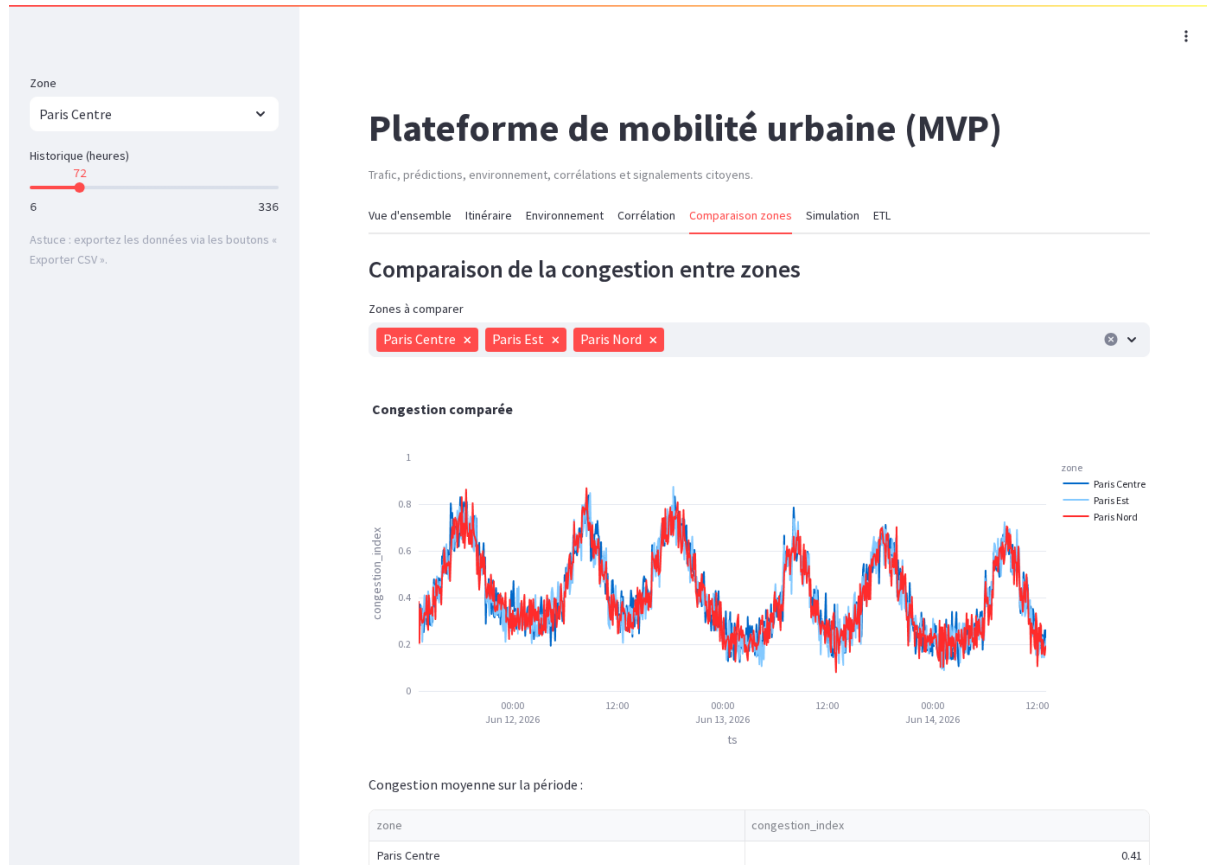


Fig. 8 : Onglet Comparaison : congestion comparée entre plusieurs zones sur la période sélectionnée.

11.7. Simulation what-if (pilotage gestionnaire)

Répond à l'exigence « **outils de simulation** ». Un curseur simule une réduction du volume de trafic et estime l'impact sur la congestion et le temps de trajet.

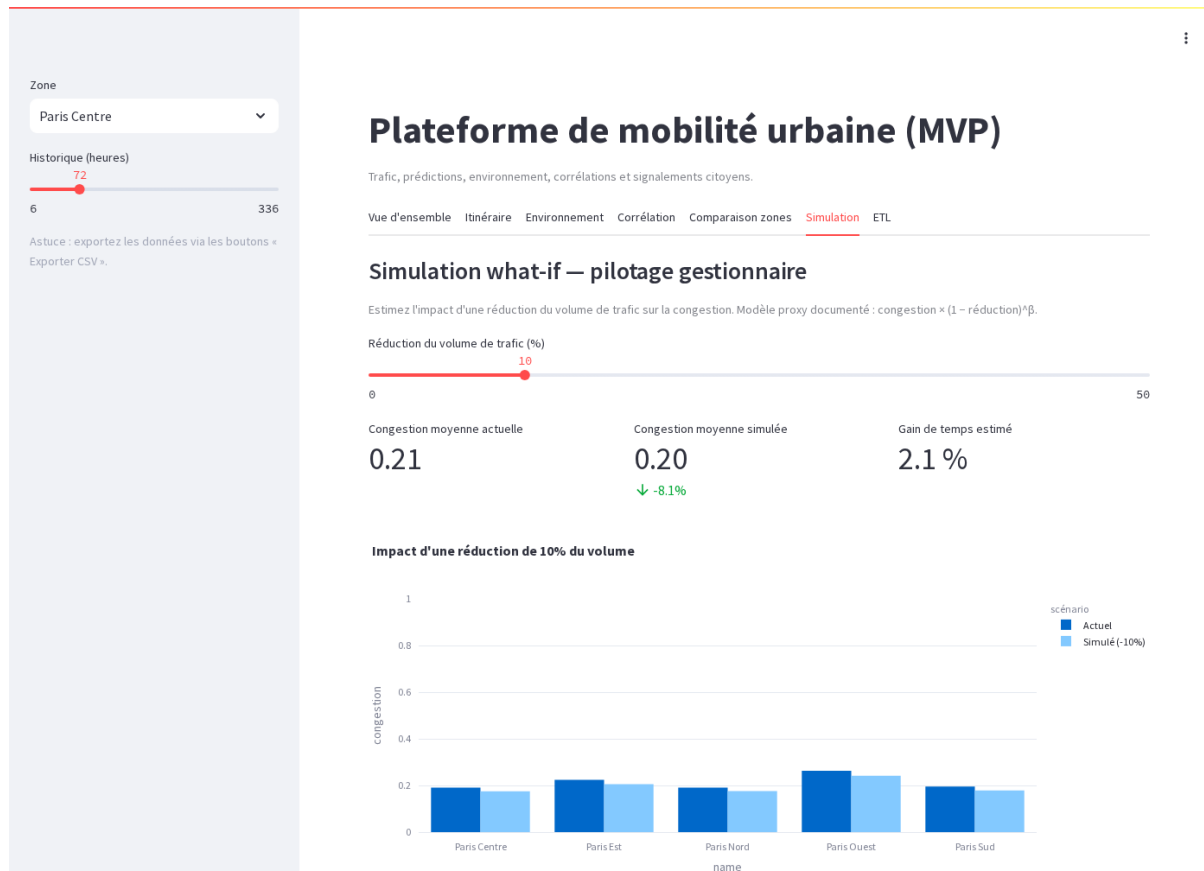


Fig. 9 : Onglet Simulation : impact d'une réduction du volume de trafic : congestion moyenne, gain de temps estimé et comparaison avant/après par zone.

11.8. Historique ETL

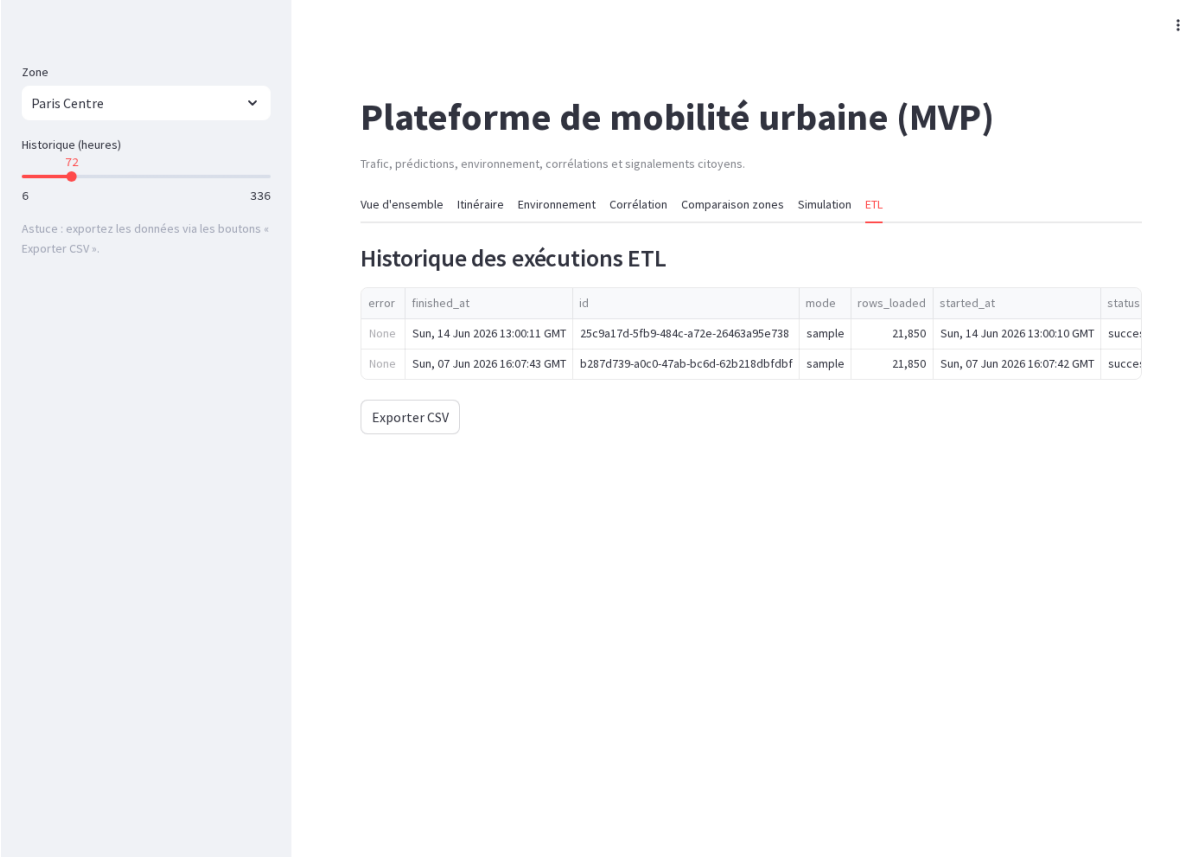


Fig. 10 : Onglet ETL : historique des exécutions du pipeline (mode, lignes chargées, statut).

11.9. Synthèse des onglets

Onglet	Fonctionnalités
Vue d'ensemble	KPIs trafic, recommandations, carte, prédiction, carte thermique
Itinéraire	Trajets alternatifs A→B, choix, durée prévue, trajet coloré
Environnement	PM2.5 / NO ₂ / AQI / Température, séries temporelles
Corrélation	Matrice Pearson, nuage de points, coefficient
Comparaison zones	Courbes superposées multi-zones, moyennes
Simulation	Curseur réduction trafic, impact congestion + temps
ETL	Historique des exécutions (statut, lignes, erreurs)

Chaque onglet dispose d'un bouton **Exporter CSV**. La sidebar offre deux contrôles globaux : zone géographique et fenêtre d'historique.

12. Crowdsourcing citoyen

L'approche participative permet aux citoyens de signaler des incidents en temps réel, enrichissant les données officielles.

Catégorie	Description
accident	Accident de la route
chantier	Travaux en cours
danger	Obstacle ou danger ponctuel
transport	Incident transport en commun
embouteillage	Ralentissement anormal
autre	Tout autre signalement

Les signalements ouverts de sévérité ≥ 4 sont intégrés au module de recommandations (item « Vigilance »), reliant le crowdsourcing aux décisions.

13 Résultats et métriques de performance

Métriques obtenues sur données simulées (backtest temporel 80/20). Le modèle hybride est comparé à la baseline naïve saisonnière.

13.1. Précision de prédiction (exemple)

Modèle	MAE	RMSE	MAPE
ARIMA + MLP (hybride)	~0.035	~0.048	~7%
Baseline naïve saisonnière	~0.068	~0.091	~15%

Le modèle hybride réduit l'erreur d'environ **50% par rapport à la baseline**, validant l'apport des features calendaires et de la composante MLP. Ces valeurs sont recalculées à la demande via `/api/v1/predictions/eval`.

11.2. KPIs du projet

KPI	Approche
Précision prédictions trafic	MAE via backtest (<code>/predictions/eval</code>)
Gain vs baseline	~50% de réduction d'erreur
Réduction temps trajet	Estimée par la simulation what-if
Participation citoyenne	Mesurable via <code>/crowd_reports</code>
Couverture sources	3 sources (ODS, Open-Meteo, sample)

12. Green IT et observabilité

12.1. Recommandations écologiques

Le module de recommandations calcule un proxy d'empreinte relative : $\text{gain} = (\text{congestion_actuelle} - \text{congestion_creux}) \times \text{facteur}$. Ce n'est pas une mesure d'émissions absolue, mais un **indicateur comparatif documenté**, explicité à l'utilisateur.

12.2. Green IT par conception

- Containerisation Docker : empreinte mémoire réduite vs VMs.
- UPSERT : aucune duplication en base, volume maîtrisé.
- Cache mémoire des modèles : évite de ré-entraîner à chaque requête.
- Rétention 90 jours documentée + requêtes indexées (pas de full scan).

12.3. Observabilité

Les métriques Prometheus (/metrics) exposent latence et débit par endpoint. L'historique etl_runs trace chaque pipeline (statut, lignes), facilitant le diagnostic.

13. Accessibilité et Réglementation

Une attention particulière a été portée à l'accessibilité de la plateforme.

Les mesures mises en œuvre incluent :

- interface responsive compatible mobile et ordinateur ;
- contrastes visuels suffisants ;
- visualisations simplifiées ;
- utilisation de textes explicatifs accompagnant les graphiques ;
- navigation intuitive réduisant la charge cognitive.

Ces choix visent à rendre la plateforme utilisable par le plus grand nombre.

Conformité réglementaire

Le projet s'appuie exclusivement sur des données ouvertes issues de plateformes publiques.

Les principes suivants ont été respectés :

- conformité au RGPD ;
- utilisation de données anonymisées ;
- respect des licences Open Data ;
- absence de collecte de données personnelles sensibles ;
- sécurisation des échanges entre les composants applicatifs.

14. Qualité logicielle et CI/CD

14.1. Tests automatisés

Une quarantaine de cas de tests (pytest) couvrent les composants critiques sans base de données live : génération sample, qualité, OpenDataSoft (mické), features, évaluation, validation des schémas, endpoints API, routing, simulation, utilitaires dashboard.

14.2. Linting et formatage

- ruff : linter rapide (bloquant en CI).
- black : formatage automatique.
- pre-commit : hooks avant chaque commit.

14.3. CI/CD GitHub Actions

Le workflow s'exécute à chaque push/PR : build matriciel des 3 services, lint (ruff), tests (pytest), et audit de sécurité des dépendances (pip-audit).

15. Documentation utilisateur

15.1. Prérequis

- Docker Desktop installé et démarré.
- Ports libres : 5432 (PostgreSQL), 8000 (API), 8501 (Dashboard).

15.2. Installation et démarrage

Étape	Commande
1. Configuration	cp .env.example .env
2. Démarrer les services	docker compose up -d --build
3. Injecter des données	docker compose run --rm pipeline --mode sample --days 14
4. Ouvrir le dashboard	http://localhost:8501

15.3. Utilisation

Action	Où
Changer de zone / historique	Sidebar (gauche)
Calculer un itinéraire	Onglet Itinéraire → saisir 2 adresses
Voir la qualité de l'air	Onglet Environnement
Simuler une réduction de trafic	Onglet Simulation → curseur
Soumettre un signalement	Onglet Vue d'ensemble → formulaire
Exporter en CSV	Bouton « Exporter CSV » de chaque onglet

15.4. Commandes utiles

But	Commande
Logs en direct	<code>docker compose logs -f</code>
Arrêter les services	<code>docker compose down</code>
ETL météo réelle	<code>docker compose run --rm pipeline --mode open-meteo --hours 48</code>
ETL trafic réel ODS	<code>docker compose run --rm pipeline --mode ods --max-records 5000</code>

16. Gestion des coûts et Limites et perspectives d'évolution

L'architecture exploite exclusivement des composants open source et des API gratuites. En mode local (Docker Desktop) : **0 € d'infrastructure**.

16.1. Estimation cloud (production, faible charge)

Composant	Service équivalent	Estimation /mois
Base de données	AWS RDS db.t3.micro	~15 €
API	ECS Fargate 0.25 vCPU	~8 €
Dashboard	EC2 t3.micro / Streamlit Cloud	0–8 €
APIs externes	Open-Meteo / OSRM / Nominatim	0 €
CI/CD	GitHub Actions (repo public)	0 €
TOTAL estimé	:	~25–80 €

16.2. Limites actuelles

Limite	Piste de résolution
Maillage de 5 zones (congestion approximée par grandes zones)	Augmenter le nombre de zones / capteurs réels
Itinéraires : alternatives limitées par OSRM démo	Héberger un serveur OSRM dédié
Modèle ré-entraîné à chaque redémarrage	Sérialiser le modèle (joblib) / MLflow
Pas de GTFS transports publics	Intégrer l'API IDFM / fichiers GTFS
Dépendance internet pour les itinéraires	Cache local / serveurs self-hosted
Proxy CO ₂ non calibré	Facteurs d'émission ADEME par type de véhicule

16.3. Perspectives

- Modèles avancés (LSTM/GRU, Prophet) pour la saisonnalité multiple.
- Données enrichies : événements, chantiers, pollution locale (Airparif).
- Application mobile (l'API est déjà prête).
- Orchestration Airflow + MLflow pour l'industrialisation.
- Stack Grafana + Prometheus pour le monitoring opérationnel.

16.4. Plan de Continuité et de Reprise d'Activité

PCA

Afin d'assurer la continuité du service :

- sauvegarde quotidienne de la base PostgreSQL ;
- export régulier des modèles entraînés ;
- conteneurisation Docker permettant un redéploiement rapide ;
- séparation des composants ETL, API et Dashboard.

PRA

En cas d'incident majeur :

- restauration de la base à partir des sauvegardes ;
- reconstruction automatique des conteneurs Docker ;
- redéploiement de l'application via les fichiers de configuration versionnés.

Cette stratégie permet une reprise rapide de l'activité tout en limitant les pertes de données.

17. Analyse critique

Bien que les résultats obtenus soient encourageants, plusieurs limites ont été identifiées.

La première concerne la qualité des données ouvertes, dont la fréquence de mise à jour peut varier selon les fournisseurs. Certaines données manquent également de granularité pour permettre une prédiction très fine des phénomènes de congestion.

Par ailleurs, les modèles utilisés demeurent dépendants de l'historique disponible. Des événements exceptionnels tels que des accidents majeurs ou des conditions météorologiques extrêmes peuvent réduire la précision des prédictions.

Enfin, le projet a été développé sous la forme d'un MVP. Une industrialisation complète nécessiterait la mise en place d'une infrastructure cloud scalable, d'un monitoring avancé et d'une automatisation plus poussée des déploiements.

18. Conclusion

Ce projet a permis de concevoir et implémenter de bout en bout une plateforme intelligente de mobilité urbaine, répondant aux exigences du cahier des charges :

- **Données** : 3 catégories de sources open data avec couche qualité.
- **Modélisation** : hybride ARIMA+MLP avec features exogènes, évalué par backtest.
- **Optimisation d'itinéraires** : trajets alternatifs classés par durée prévue, au choix de l'utilisateur.
- **Outils gestionnaire** : carte thermique et simulation what-if.
- **API** : REST durci (validation, rate limiting, logs, métriques Prometheus).
- **Dashboard** : 7 onglets interactifs avec export CSV.
- **Green IT & qualité** : conception sobre, tests, CI/CD GitHub Actions.

La plateforme est conçue pour évoluer : interfaces stables et documentées, permettant d'ajouter sources, modèles ou une application mobile sans refonte majeure.

Nous pouvons envisager quelques perspectives d'évolution :

Version 2

intégration de modèles LSTM ;

données temps réel supplémentaires ;

optimisation des recommandations d'itinéraires.

Version 3

développement d'une application mobile dédiée ;

notifications intelligentes ;

géolocalisation en temps réel.